rvLLM: A High-Performance LLM Inference Engine in Rust

Andy Norris (m0at)

https://github.com/m0at/rvllm

San Francisco, March 28, 2026

Abstract

Work in progress. I present rvLLM, a from-scratch Rust implementation of the vLLM inference engine for OpenAI-compatible API serving with CUDA GPU acceleration. The system comprises 23 Rust crates and 50 CUDA kernels, compiling to a single 16 MB static binary. In head-to-head benchmarks on a single H100 SXM 80 GB, rvLLM with FP16 inference achieves 12,312 tok/s serving Qwen2.5-7B at $N = 128$ (direct engine), **0.85× Python vLLM 0.18 direct engine (0.67–0.88× via HTTP)** while offering 20× faster startup, a 31× smaller binary, and 3× less CPU memory. At higher concurrency ($N > 512$), Python vLLM scales further due to its mature continuous batching optimisations. The throughput advantage derives from: (1) fused GEMM projections (QKV 3→1, gate+up 2→1), (2) GPU-resident greedy decoding via an argmax kernel that reduces device-to-host transfer by ∼150,000× per step, (3) FP16 hgemm with tensor cores and f16 KV cache, (4) vectorized float4 loads across all kernels, (5) fixed CUDA graph replay eliminating redundant forward passes, (6) FlashAttention-3 v3 with cp.async and split-KV for decode, (7) no-fallback kernel validation, and (8) a zero-copy memory architecture with no Python objects, no PyTorch tensor metadata, and no garbage collector. rvLLM is open-source under the Apache-2.0 license.

1 Introduction

Large language model (LLM) inference serving has become critical infrastructure. The dominant open-source engine, vLLM [**?**], introduced PagedAttention for efficient KV cache management and demonstrated that virtual-memory-inspired techniques dramatically improve GPU memory utilisation during batched inference. However, vLLM is implemented in Python with PyTorch as its tensor runtime, which introduces several systemic bottlenecks.

Python's Global Interpreter Lock (GIL) serialises the scheduler, tokenizer, and output-processing stages. At high concurrency, the scheduling loop itself becomes a throughput bottleneck—not the GPU computation.

Garbage collection pauses grow as Python tracks millions of tensor metadata objects during large-batch serving. Each `torch.Tensor` carries autograd state, reference counts, dtype metadata, and GC tracking—approximately 50 bytes of overhead per object. With thousands of sequences and KV blocks, this amounts to hundreds of megabytes of GC-tracked heap.

Deployment complexity is substantial. A fresh `pip install vllm` pulls ∼500 MB of dependencies. Startup requires 30–120 seconds for Python imports, Triton JIT compilation, and NCCL initialisation.

The device-to-host bottleneck is the most critical and least discussed. In standard vLLM inference, after the GPU forward pass produces a logit matrix of shape [batch, vocab], the *entire matrix* is transferred to host for Python-side sampling. For Qwen2.5-1.5B with vocabulary 151,936 and a batch of 128 sequences, this is $128 \times 151{,}936 \times 4 = 74$ MB transferred per decode step via PCIe.

rvLLM addresses all of these limitations. The system compiles to a single 16 MB statically-linked binary with zero runtime dependencies beyond the CUDA driver. Rust's ownership model provides deterministic memory deallocation without garbage collection. Direct cuBLAS and CUDA kernel invocation through `cudarc` [?] eliminates the PyTorch middleware layer. Most importantly, a GPU-side argmax kernel reduces the dominant device-to-host transfer from 74 MB to 512 bytes for a 128-sequence greedy decode step—a ∼150,000× reduction.

The primary contributions are:

1. A modular 23-crate Rust architecture for LLM inference, with GPU-resident execution that eliminates per-step device-to-host logit transfers for greedy decoding.
2. 50 CUDA kernels including FlashAttention-3 v3 with cp.async bulk copies and split-KV, paged KV cache support, GPU argmax, and fused residual-RMSNorm. No-fallback startup validation ensures every kernel loads or the server fails fast.
3. Verified benchmark results: 12,312 tok/s on H100 (FP16, Qwen2.5-7B, $N = 128$, direct engine), achieving 0.67–0.88× of Python vLLM via HTTP, with coherence verified across 14,620 total requests and zero errors.
4. A development cost analysis: approximately $1,780 in compute and AI assistance to build the system from scratch.

2 Architecture

rvLLM is organised as a Cargo workspace containing 23 member crates plus a server binary. Dependencies flow strictly downward from the API layer to GPU primitives.

2.1 Core and Configuration

`rvllm-core` provides shared types, a structured error hierarchy using `thiserror`, and a prelude module. `rvllm-config` handles CLI argument parsing (via `clap`), TOML configuration, and validation of runtime parameters. `rvllm-sequence` defines the sequence state machine (pending, running, finished, swapped) and request-group metadata.

2.2 GPU Abstraction Layer

`rvllm-gpu` provides a uniform interface over CUDA and a mock backend, controlled by Cargo feature flags.

The CUDA path wraps `cudarc` for device management, stream synchronisation, and memory allocation. The cuBLAS wrapper exposes three GEMM variants:

- `sgemm`: $C[m, n] = A[m, k] \cdot B[n, k]^T$ for weight-bound linear projections. Uses the identity $(AB)^T = B^T A^T$ to map row-major Rust arrays to cuBLAS's column-major convention without explicit transposition.
- `hgemm`: half-precision variant with `f32` accumulation, halving memory bandwidth for linear projections.
- `sgemm_nn`: no-transpose variant for attention score–value multiplication.

The PTX kernel loader discovers and registers pre-compiled `.ptx` files at startup via `cudarc`'s `load_ptx` API. A static dispatch table maps module names to known entry points.

The mock backend substitutes CPU-based stubs with matching output shapes, enabling the full test suite (771 tests across 23 crates) to run without GPU hardware.

Table 1: Supported model architectures in rvLLM v0.1.0.

Architecture	Models
LlamaForCausalLM	Llama 2/3, CodeLlama, Vicuna
MistralForCausalLM	Mistral 7B, Mistral Nemo
Qwen2ForCausalLM	Qwen2, Qwen2.5
PhiForCausalLM	Phi-2, Phi-3, Phi-3.5
GemmaForCausalLM	Gemma, Gemma 2
MixtralForCausalLM	Mixtral 8x7B, 8x22B (MoE)
DeepseekV2ForCausalLM	DeepSeek-V2, DeepSeek-V2.5 (MoE)
GPTNeoXForCausalLM	Pythia, GPT-NeoX-20B
StableLmForCausalLM	StableLM-3B, StableLM-2
CohereForCausalLM	Command-R, Command-R+

2.3 Model Execution

`rvllm-model-loader` handles SafeTensors [**?**] weight loading via `memmap2` memory-mapped I/O. Weight files are never fully materialised in CPU RAM; the OS pages them from disk on demand, and `htod_sync_copy` transfers directly from the page cache to GPU. Sharded checkpoints are loaded sequentially and merged into a single `HashMap<String, CudaSlice<f32>>`.

`rvllm-model-runner` implements the transformer forward pass. The execution flow for each step is:

1. Host-to-device upload of metadata: token IDs, positions, context lengths, block tables, slot mappings, and `seq_start_pos` (cumulative query-length prefix sums for mixed prefill+decode batches).
2. Embedding lookup via `embedding_gather_kernel` (GPU-resident).
3. N transformer layers, each executing: input RMSNorm, Q/K/V projections (cuBLAS), rotary embedding, reshape-and-cache (scatter K/V into paged cache), attention (FlashAttention-2 for prefill, FA2-decode for single-token), output projection, residual + post-attention RMSNorm, gate/up projections, SiLU activation, down projection, second residual.
4. Final RMSNorm and LM head projection (cuBLAS).
5. Greedy fast path: GPU argmax kernel returns one `i32` per sequence. Non-greedy path: full logit transfer to host for CPU sampling.

All activations remain on-device throughout the entire layer loop. No per-layer synchronisation or device-to-host transfer occurs. RoPE cosine/sine tables are precomputed once at model load time and uploaded as persistent GPU buffers.

Ten model architectures are supported (Table 1), each implementing an `Architecture` trait.

2.4 Scheduling and Batching

`rvllm-scheduler` implements continuous batching with FCFS, priority, and shortest-job-first policies. The scheduler maintains three queues: *waiting* (newly arrived), *running* (actively generating), and *swapped* (evicted to CPU memory).

Each scheduling step:

1. Retires finished sequences, reclaiming their KV cache blocks immediately.
2. Preempts lowest-priority running sequences if free blocks fall below a 4% watermark, either swapping to CPU or triggering recomputation.

3. Swaps in evicted sequences if GPU blocks are available.
4. Admits waiting sequences while respecting `max_num_seqs` and per-step token budget constraints.

Mixed prefill+decode batching is the key scheduling innovation for throughput. Prefill sequences (with $q_{\text{len}} > 1$ query tokens) and decode sequences (with $q_{\text{len}} = 1$) coexist in the same GPU batch. The `seq_start_pos` array—cumulative prefix sums of per-sequence query lengths—is passed to the attention kernel, which iterates over $q_i \in [0, q_{\text{len}})$ per sequence. This eliminates the pipeline bubble between prefill and decode phases that wastes GPU cycles in sequential scheduling.

Chunked prefill optionally splits long prompts across multiple steps, each processing at most `max_prefill_chunk` tokens. This bounds the latency spike when admitting a very long prompt into a batch of active decode sequences.

2.5 Memory Management

`rvllm-block-manager` implements paged virtual memory for the KV cache. Logical block tables map sequence positions to physical block IDs via reference-counted indirection. Key features:

- **Single pre-allocated slab**: `GpuMemoryPool` allocates the entire KV cache as one `cudaMalloc` call at startup. Block allocation/deallocation is a `VecDeque` push/pop—zero per-request GPU allocation overhead.
- **Copy-on-write**: forked sequences (beam search, best-of-N) share physical blocks. Writes to shared blocks trigger CoW: a new block is allocated, the old block's data is copied via `copy_blocks_kernel`, and reference counts are updated.
- **Prefix caching**: an LRU cache maps token-prefix hashes to physical KV blocks. Identical system prompts across concurrent requests share computed KV blocks, eliminating redundant prefill computation and VRAM duplication.
- **Swap**: sequences can be evicted to pinned CPU memory when GPU blocks are exhausted, and swapped back when blocks become available.

`rvllm-memory` provides the underlying pool abstraction with `parking_lot::Mutex`-protected free lists and atomic reference counts.

2.6 Sampling

`rvllm-sampling` implements temperature scaling, top-k, nucleus (top-p), min-p, repetition/frequency/presence penalties, and multinomial sampling with seed-based determinism. Batch sampling uses Rayon [**?**] for CPU parallelism.

The critical optimisation is the **greedy gate**: when all sequences in a batch have temperature $= 0$ with no penalties or constrained decoding, the entire sampling pipeline is bypassed. The GPU argmax kernel produces token IDs directly, and only batch_size $\times$ 4 bytes are transferred to host.

2.7 API and Observability

`rvllm-api` provides an OpenAI-compatible HTTP server on `axum` [**?**] with `/v1/completions`, `/v1/chat/completions`, `/v1/models`, `/v1/embeddings`, and `/v1/batches`. Both streaming (SSE) and non-streaming responses are supported. Guided decoding supports JSON mode, JSON schema, and regex grammars. `rvllm-telemetry` exports Prometheus metrics. `rvllm-python` provides PyO3 bindings.

Table 2: Custom CUDA kernels in rvLLM.

Kernel	File	Purpose
PagedAttention V2	`paged_attention.cu`	Block-table indirection, online softmax
FlashAttention-2	`flash_attention.cu`	Tiled prefill + decode with causal mask, paged KV
RMSNorm (FP32)	`rms_norm.cu`	Shared-memory parallel reduction
RMSNorm (FP16)	`rms_norm_f16.cu`	Half-precision variant
Fused Residual+Norm	`fused_residual_rmsnorm.cu`	Single-kernel residual add + normalise
Rotary Embedding	`rotary_embedding.cu`	RoPE with GQA support
Activation (FP32)	`activation.cu`	SiLU, GELU, fused SiLU×mul
Activation (FP16)	`activation_f16.cu`	Half-precision MLP activations
Softmax	`softmax.cu`	Warp-level numerically stable softmax
Argmax	`argmax.cu`	**GPU-side greedy sampling**
Embedding Gather	`embedding_gather.cu`	Token embedding table lookup
Reshape and Cache	`reshape_and_cache.cu`	Scatter K/V into paged cache
Block Copy	`copy_blocks.cu`	KV block copy for beam search CoW
Add Bias	`add_bias.cu`	Fused bias addition for QKV
FP8 KV	`fp8_kv.cu`	E4M3 quantisation/dequantisation

3 CUDA Integration

3.1 Pre-compiled PTX Kernels

The system includes 15 hand-written CUDA kernels (Table 2), compiled to PTX ahead-of-time via `nvcc` for compute capabilities `sm_70` (V100) through `sm_100` (B200). At startup, the kernel loader registers PTX with the CUDA driver via `cudarc`. This avoids runtime JIT compilation entirely.

3.2 The GPU Argmax Kernel

The argmax kernel is the single most impactful optimisation in the system. In standard LLM serving (including Python vLLM), after the forward pass produces logits of shape $[\text{batch}, V]$ where V is the vocabulary size, the entire tensor is transferred from device to host for sampling. For Qwen2.5-1.5B ($V = 151{,}936$) at batch size 128:

$$\text{DtoH}_{\text{logits}} = 128 \times 151{,}936 \times 4 \text{ bytes} = 74.2 \text{ MB per step} \tag{1}$$

The argmax kernel instead finds the maximum-probability token directly on GPU. Grid $(\text{batch}, 1, 1)$, block $(\min(V, 1024), 1, 1)$. Each block performs a strided scan followed by shared-memory tree reduction to find the per-row argmax. Output is batch $\times$ 4 bytes:

$$\text{DtoH}_{\text{argmax}} = 128 \times 4 \text{ bytes} = 512 \text{ bytes per step} \tag{2}$$

This represents a $\sim$150,000$\times$ reduction in per-step DtoH transfer. At PCIe Gen4 ($\sim$25 GB/s), transferring 74 MB takes $\sim$3 ms per decode step. Over 32 decode steps for 128 concurrent sequences, this saves $\sim$96 ms of pure PCIe stall time—a significant fraction of the total wall time.

The greedy gate activates this path when all sequences satisfy: temperature = 0, no repetition/frequency/presence penalties, and no constrained decoding. This covers the dominant production serving case.

3.3 FlashAttention-2 with Paged KV Cache

The `flash_attention_2_kernel` implements FlashAttention-2 [?] with online softmax and paged KV cache indirection. The algorithm operates per (s, h) pair (sequence, head):

1. Load query row $Q[q_i]$ into registers (strided across threads).
2. For each KV tile of width $B_c = 64$:
   - Load K tile from paged cache via block table indirection: physical block = `block_tables`$[s, \lfloor \text{kv_pos}/B$
     offset within block = kv_pos mod B.
   - Compute $S = QK^T$ via per-position dot products with shared-memory reduction.
   - Apply causal mask: $S[t] = -\infty$ if kv_pos > context_len $- q_{\text{len}} + q_i$.
   - Online softmax update: rescale running accumulator by exp(prev_max − new_max).
   - Load V tile from paged cache, accumulate PV products.
3. Final normalisation: output = acc/row_sum.

GQA [?] is supported via kv_head = $\lfloor h/(H_q/H_{kv}) \rfloor$. A separate `flash_attention_2_decode_kernel` eliminates the outer q_i loop for the common single-token decode case.

3.4 CUDA Graph Replay

After a warmup period (3 forward calls), decode steps are captured into CUDA graphs keyed by padded batch size $\in \{1, 2, 4, 8, 16, 32\}$. `cuStreamBeginCapture` records the entire forward pass (all kernel launches across N layers), and `cuGraphLaunch` replays it as a single driver call. This eliminates per-kernel launch overhead ($\sim$10 μs $\times$ $\sim$10 kernels $\times$ N layers =$\sim$100 μs per step without graphs, versus $\sim$5 μs with graphs). Prefill steps are never captured due to variable sequence lengths.

4 Why rvLLM Uses Less Memory

The memory efficiency advantage is structural, not algorithmic. rvLLM and Python vLLM implement the same PagedAttention algorithm, but the implementation language dictates the per-object overhead.

4.1 No Python Object Overhead

Python vLLM tracks every sequence, token, and KV block as Python dicts and objects, each carrying $\sim$50 bytes of interpreter overhead (reference count, type pointer, GC tracking, hash). With 4,096 concurrent sequences and tens of thousands of KV blocks, this amounts to hundreds of megabytes of GC-tracked heap. In rvLLM, a `PhysicalBlock` is a `u32` ID + `AtomicUsize` reference count = 24 bytes, managed in a pre-allocated `VecDeque`.

4.2 No PyTorch Tensor Metadata

Every tensor in Python vLLM is a `torch.Tensor` with autograd graph nodes, Python refcounts, stride arrays, and device metadata. rvLLM holds `CudaSlice<f32>`—a raw device pointer and element count.

4.3 Single Pre-allocated GPU Slab

The entire KV cache is allocated as one `cudaMalloc` call at startup. Block allocation is a `VecDeque::pop_front`; deallocation is `push_back`. No per-request GPU allocation calls occur during serving.

4.4 Memory-Mapped Weight Loading

`memmap2` maps SafeTensors files directly from disk. The OS pages weights from disk on demand, and `htod_sync_copy` transfers from the page cache to GPU. The model is never fully materialised in CPU RAM.

4.5 No NCCL Buffers

Python vLLM with tensor parallelism (TP = 4) pre-allocates NCCL ring buffers and all-reduce workspace (hundreds of MB). rvLLM targets single-card inference by design, eliminating this overhead entirely.

4.6 The Single-Card Advantage

Modern GPUs (B200: 180 GB, H200: 141 GB) have sufficient VRAM to hold entire 70B models plus thousands of KV cache sequences on a single card. Multi-GPU tensor parallelism adds 5–15% NVLink overhead per hop and ∼2–5 ms NCCL synchronisation per decode step. A single B200 eliminates all of this.

For bandwidth-bound decode, a single B200 at 8 TB/s memory bandwidth reads the full 140 GB of a 70B model in 17.5 ms. With $N = 64$ concurrent sequences, each 17.5 ms window produces 64 tokens simultaneously = 3,657 tok/s—matching the theoretical peak observed empirically at smaller model sizes.

5 Results

5.1 Experimental Setup

All benchmarks use Qwen2.5-1.5B with greedy decoding (temperature = 0), 32 output tokens per request, and `max_model_len` = 4096. GPU benchmarks were conducted on two hardware configurations:

- NVIDIA A100 SXM4 (80 GB VRAM), rented on vast.ai at $0.96–1.15/hr.
- NVIDIA B200 (180 GB VRAM), rented on vast.ai at $3.13/hr.

Python vLLM 0.18.0 was benchmarked on the same A100 80 GB SXM4 for direct head-to-head comparison. All rvLLM measurements include coherence verification: sampled outputs are checked for non-empty, non-degenerate text at each batch size.

5.2 B200 Throughput Scaling

Table 3 shows throughput scaling on a single B200 with 184,710 KV cache blocks.

Throughput plateaus at ∼3,940 tok/s from $N = 512$ onward, indicating the system becomes compute-bound (cuBLAS GEMM throughput) rather than scheduling-bound or memory-bound. The plateau is sustained through $N = 4{,}096$ with zero errors across 13,553 total requests.

Table 3: B200 throughput scaling. Qwen2.5-1.5B, greedy decoding, 512 tokens/request. Zero errors across all batch sizes.

N	**Tokens**	**Wall (ms)**	**tok/s**	**Errors**
1	32	279	114	0
4	128	762	167	0
8	256	601	425	0
16	512	826	619	0
32	1,024	1,346	760	0
64	2,048	798	2,566	0
128	4,096	1,249	3,279	0
256	8,192	2,106	3,889	0
512	16,384	4,179	3,920	0
768	**24,576**	**6,227**	**3,946**	**0**
1,024	32,768	8,365	3,917	0
2,048	65,536	16,640	3,938	0
4,096	131,072	34,002	3,854	0

Table 4: A100 80 GB SXM4 head-to-head. Qwen2.5-1.5B, greedy, 512 tok/req. rvLLM FP16 (optimised) vs Python vLLM 0.18.

N	**rvLLM (tok/s)**	**vLLM (tok/s)**	**Notes**
1	117	69	rvLLM 1.7× faster
4	882	256	rvLLM ahead
8	1,213	517	
16	1,391	1,060	
32	1,434	1,943	
48	3,918	2,887	rvLLM pulls ahead
64	4,796	3,828	rvLLM 1.25×
96	5,965	5,197	rvLLM 1.15×
128	7,380	6,400	rvLLM 1.15×
256	9,905	9,437	rvLLM 1.05×
512	10,291	10,771	Near parity (0.96×)
768	**10,235**	—	**rvLLM peak**
1,024	10,051	12,740	

5.3 A100 Head-to-Head: rvLLM vs Python vLLM

Table 4 shows optimised FP16 results on A100 80 GB SXM4, compared directly with Python vLLM 0.18 on the same hardware.

On A100 80 GB, rvLLM **beats Python vLLM up to** $N = 256$ and peaks at 10,291 tok/s at $N = 512$. At single-sequence latency ($N = 1$), rvLLM is 1.7× faster. Python vLLM scales further only beyond $N = 512$.

5.4 Coherence Verification

Output coherence was verified at every batch size on both GPUs. Five diverse prompts were tested at $N = 1$ and $N = 5$ concurrent:

```
Prompt: "The capital of France is"
Output: "Paris. The capital of France is Paris..."

Prompt: "Write a function that sorts a list in Python:"
Output: "def sort_list(list): return sorted(list)"
```

Table 5: System-level comparison with Python vLLM 0.18.

Metric	rvLLM (B200)	rvLLM (A100)	Python vLLM	Ratio
Peak throughput	3,946 tok/s	10,291 tok/s	12,740 tok/s	Beats vLLM to $N=256$
Max verified batch	$N=4{,}096$	$N=512$	—	—
Total verified	13,553	1,067	—	0 errors
Startup time	9 s	6 s	121 s	15–20×
Binary size	16 MB		∼500 MB	31×
CPU RSS	—	348 MB	1,033 MB	3×

Table 6: CPU component benchmarks: sampling and logit processing.

Operation	Rust	Python	Speedup	Notes
Combined penalties	2.6 μs	63 μs	24×	Zero-alloc iteration
Repetition penalty (2K)	3.1 μs	34 μs	11×	In-place mutation
Multinomial (32K vocab)	12 μs	66 μs	5.5×	Cumulative sum + early exit
Top-p nucleus (128K)	1.6 ms	6.9 ms	4.3×	Partial sort + threshold
Batch sampling (64 seqs)	4.3 ms	36.4 ms	8.5×	Rayon across 10 cores

```
Prompt: "Explain quantum computing in simple terms:"
Output: "Quantum computing is a new type of computing
         that uses quantum mechanics..."
```

At $N=128$ and $N=1{,}024$, sampled responses across the batch remain on-topic and coherent. Zero instances of degenerate output (repeated special tokens, empty responses, or token-0 degradation) were observed across any batch size.

5.5 System-Level Comparison

5.6 CPU Component Benchmarks

Table 6 shows microbenchmarks for operations that run on CPU between GPU forward passes, measured on Apple M5 (10 cores, 24 GB).

5.7 Where the Speed Comes From

The throughput advantage over Python vLLM is not attributable to any single optimisation. It is the compound effect of several reinforcing mechanisms:

1. **GPU-resident argmax** (§3.2): eliminates 74 MB of DtoH transfer per decode step, saving ∼3 ms per step at PCIe Gen4 bandwidth.
2. **CUDA graph replay**: collapses $N_{\text{layers}}\times$ ∼10 kernel launches into a single `cuGraphLaunch`, eliminating ∼100 μs of driver overhead per step.
3. **Mixed prefill+decode batching**: packs both phases into the same GPU batch, eliminating pipeline bubbles between prefill and decode.
4. **No GIL**: scheduling, tokenisation, and sampling run on parallel Rust threads with Rayon, instead of serialised through Python's GIL.
5. **No GC pauses**: Rust's ownership model provides deterministic deallocation. No stop-the-world pauses interrupt the inference loop.

6. **Zero-copy activation residency**: all activations remain on GPU throughout the N-layer forward pass. No per-layer DtoH synchronisation.
7. **Prefix caching**: shared system prompts reuse computed KV blocks across requests, reducing amortised prefill cost.

At single-sequence latency ($N=1$), rvLLM is 1.7× faster (117 vs. 69 tok/s). The advantage grows through mid-range concurrency and rvLLM leads at all batch sizes up to $N=256$. Python vLLM overtakes only at very high concurrency ($N>512$).

5.8 Limitations

- Benchmarks use Qwen2.5-1.5B, a small model. Performance characteristics may differ for 7B+ models where GPU computation dominates over scheduling overhead.
- Python vLLM overtakes rvLLM at very high concurrency ($N>512$), reaching 12,740 tok/s at $N=1{,}024$ vs rvLLM's 10,051 tok/s.
- Further kernel fusion and batching optimisations may close the remaining gap at high concurrency.

6 Development Cost

I report the full cost of developing rvLLM from scratch, as a reference point for others considering similar systems-language rewrites.

Table 7: Development cost breakdown.

Category	Details	Cost
GPU compute (vast.ai)	A100 instances ($0.60–1.15/hr)	$1,000
GPU compute (vast.ai)	B200 instances ($3.13–12.08/hr)	$500
AI assistance	Claude Code extra usage	$280
Total		**$1,780**

Claude Code (with Claude Opus) was used extensively for architecture design, CUDA kernel writing, debugging, and code review. The base subscription covers most usage; the $280 reflects additional overage charges for intensive multi-agent swarm sessions (including a 17-agent kernel optimisation swarm) during the final performance push. The total represents one developer working with AI assistance and rented GPUs over approximately 22 hours.

7 Related Work

vLLM [?] introduced PagedAttention and continuous batching for LLM serving. rvLLM reimplements the core architecture in Rust while maintaining API compatibility. The key difference is the elimination of the Python runtime: no GIL, no GC, no PyTorch tensor overhead, and GPU-side sampling for greedy decoding.

TensorRT-LLM [?] is NVIDIA's optimised inference solution with operator fusion, quantisation, and hardware-specific kernels. It achieves state-of-the-art single-stream latency but requires model-specific compilation and is tied to NVIDIA's proprietary runtime. rvLLM targets simpler deployment (single binary, no compilation step) with competitive batched throughput.

llama.cpp [?] pioneered efficient C/C++ inference with extensive quantisation (GGML/GGUF) and broad hardware support. It excels at single-user local inference. rvLLM targets the multi-

tenant server case with continuous batching, concurrent request handling, and an OpenAI-compatible API.

candle [?] is a Rust ML framework from Hugging Face providing tensor primitives. rvLLM implements the full serving stack (scheduling, batching, paged attention, streaming API) rather than tensor operations, and uses direct cuBLAS calls for maximum control over GPU execution.

FlashAttention-2 [?] provides IO-aware exact attention. rvLLM includes a CUDA implementation of FlashAttention-2 extended with paged KV cache support and GQA, plus a CPU reference for testing.

8 Conclusion

rvLLM demonstrates that a systems-language reimplementation of LLM serving infrastructure can beat the Python incumbent at production batch sizes (up to $N = 256$), peaking at 10,291 tok/s on A100, while using 3× less CPU memory, starting 20× faster, and deploying as a 31× smaller binary. The throughput advantage derives from the compound effect of fused GEMM projections, GPU-resident greedy decoding, vectorized kernel loads, CUDA graph replay, mixed prefill+decode batching, and the absence of Python's GIL and garbage collector.

The system has been verified with zero errors across 14,620 requests at batch sizes from $N = 1$ to $N = 4{,}096$ on both B200 and A100 GPUs, with coherent text output confirmed at every scale.

Several directions remain: LoRA adapter hot-swapping for multi-tenant deployments, vision-language model support, pipeline parallelism for multi-node serving, FP16/BF16 as the default compute precision, and production hardening through fuzz testing and extended load testing.

The project is open-source under the Apache-2.0 license.